



Building

Describe the methods to ensure the delivery of quality code, such as coding standards, pull requests, code review, and static code analysis.

Develop your career with Salesforce Training and Certification Preparation

Table of Contents

-  [Coding Standards](#)
-  [Pull Requests](#)
-  [Code Review](#)
-  [Static Code Analysis](#)
-  [Business Scenario](#)
-  [Requirements & Solutions](#)



After studying this topic, you should be able to:

-  Describe the use of coding conventions and standards to add consistency to code.
-  Describe how pull requests can be utilized to improve code quality.
-  Describe the use of code reviews to catch bugs and ensure code quality and adherence to best practices.
-  Describe how to use static code analysis tools to conduct automated code checks.
-  Describe the methods that can be used to ensure the delivery of quality code for various business requirements.



Introduction

There are various methods that can be utilized to ensure the delivery of **quality code** in Salesforce. These include the creation of **coding conventions and standards**, the integral process of **pull requests** in Git-based workflows, the practice of **code review** for quality assurance, and the technique of **static code analysis** for the examination of code. These methods can be used to improve code consistency, enhance code security, catch bugs, and significantly improve the overall integrity of the codebase.



Ensuring Code Quality

Coding Standards

Coding conventions/standards are guidelines for styling and structuring code to add consistency and ensure team adherence to the same principles.



Pull Requests

In Git-based workflows, a pull request is a proposal to merge changes from one branch into another.



Code Review

Code review is a quality assurance activity in which one or more people check the source code to catch bugs and ensure code quality and adherence to best practices.



Static Code Analysis

Static code analysis examines the code without executing it to identify potential vulnerabilities, coding errors, and adherence to coding standards.



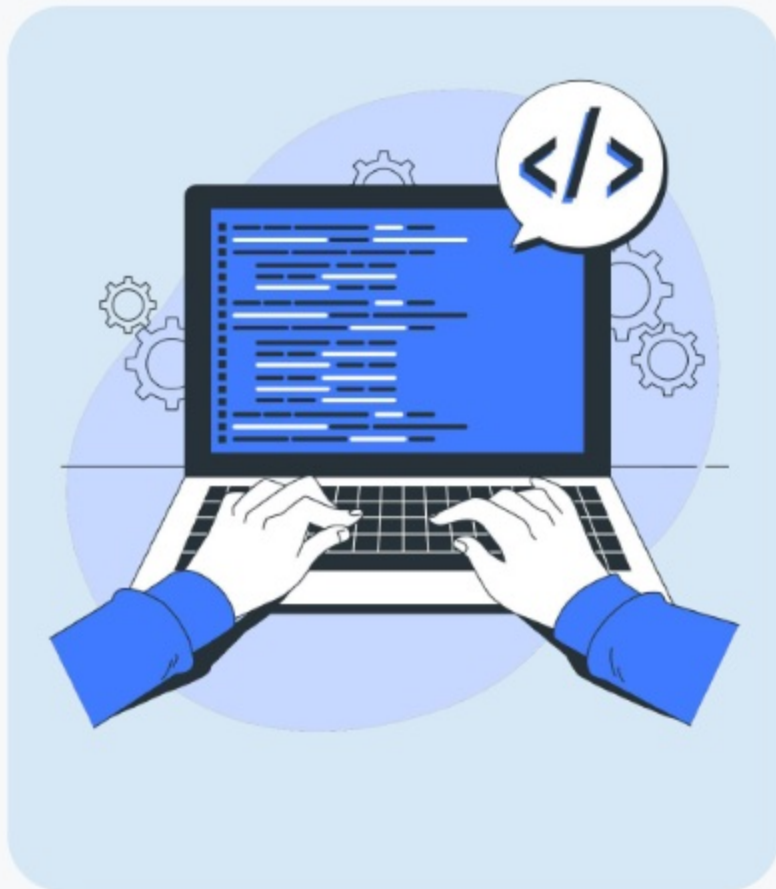
Coding Standards

[Table of Contents](#)



Coding Standards

Coding conventions/standards are **guidelines for styling and structuring code** to add consistency and ensure team adherence to the same principles.



❖ CODE QUALITY

Coding conventions help maintain the **clarity and quality of code** in Salesforce orgs as they grow and become more complex.

❖ CONVENTIONS

Coding conventions cover **naming conventions for Salesforce elements, frameworks** for making code **reusable and maintainable**, and **best practices for Apex coding**.

❖ BENEFITS

Benefits of coding conventions include **standardization, easier debugging and maintenance, fewer bugs**, and ultimately, **higher quality Salesforce solutions** leading to increased customer satisfaction.

Coding Standards

Coding conventions/standards are **guidelines for styling and structuring code** to add consistency and ensure team adherence to the same principles.



◆ TEAM AGREEMENT

Implementing coding conventions requires **no specific tools or software**, just **team agreement** and **commitment** to follow these guidelines.

◆ SUCCESS CLOUD CONVENTIONS

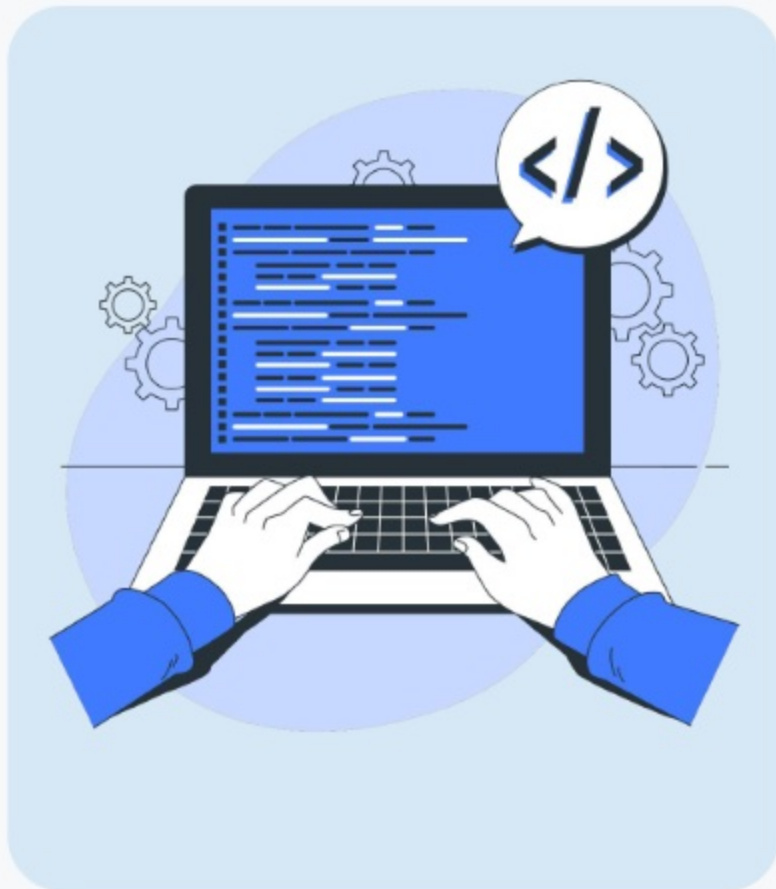
Success Cloud Naming Conventions offer guidelines for **naming common Salesforce entities**, aiming to **standardize component identification**.

◆ APEX CLASSES

For Apex classes, **naming conventions** suggest using **PascalCase**, including a **namespace prefix** (if applicable), the **class name**, and an **optional suffix** indicating the class type (e.g., Controller, Extension). For example, SCT_NewOrderController.

Coding Standards

Coding conventions/standards are **guidelines for styling and structuring code** to add consistency and ensure team adherence to the same principles.



❖ FRAMEWORK

A framework is a **reusable, highly optimized structure** that acts as a **building block**, offering common functionality for developers to adapt to their needs, enhancing **development speed**, **code clarity**, and **debugging**.

❖ TRIGGER FRAMEWORK

A trigger framework is essential for **structured, efficient trigger management** in Salesforce.

❖ GOOD TRIGGER FRAMEWORK

A good trigger framework is **scalable, traceable, reusable, atomic** (ability to bypass specific triggers), and **optimized** (minimal SOQL calls, prevention of recursion, and shared result sets).

Coding Standards

Coding conventions/standards are **guidelines for styling and structuring code** to add consistency and ensure team adherence to the same principles.



❖ RECOMMENDED TRIGGER FRAMEWORK

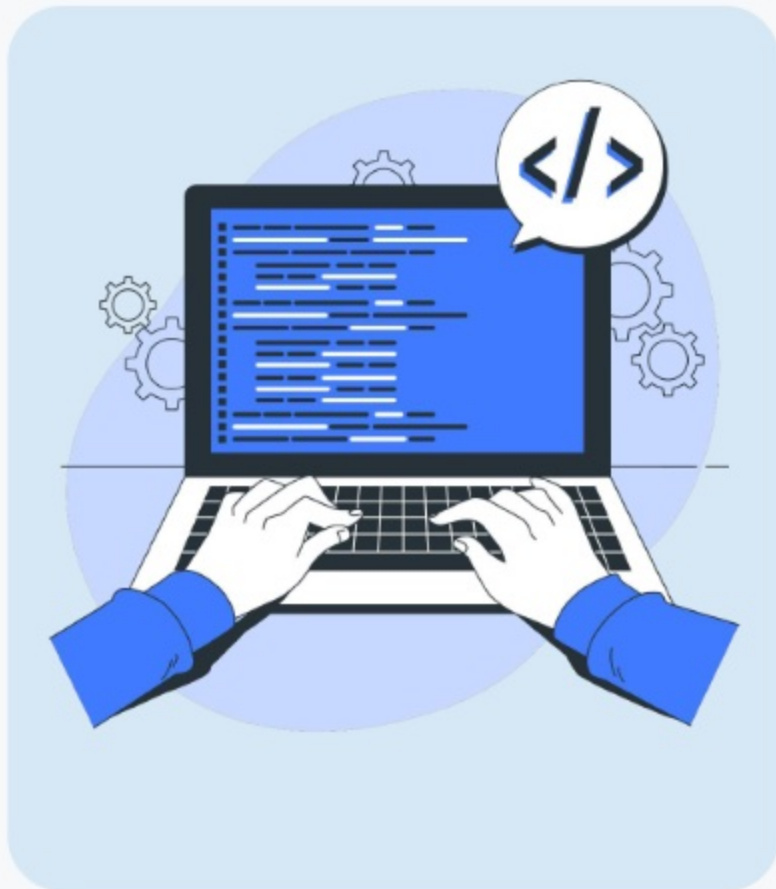
Kevin O'Hara's **SFDC Trigger Framework** is recommended for **large-scale projects**. It features an abstract TriggerHandler base class to handle execution and prevent recursion, with child classes overriding specific trigger context methods.

❖ ERROR LOGGING FRAMEWORK

An error logging framework **records errors in a structured manner**, aiding in debugging and maintaining Salesforce orgs. Implementations should log errors to a custom object, enabling actions such as email notifications or task creation based on errors.

Coding Standards

Coding conventions/standards are **guidelines for styling and structuring code** to add consistency and ensure team adherence to the same principles.



❖ BYPASS PATTERNS

Hierarchical custom settings or **custom permissions** can be utilized to temporarily disable configurations (like validation rules or triggers) for specific users, profiles, or the entire org.

❖ TEST DATA FACTORY

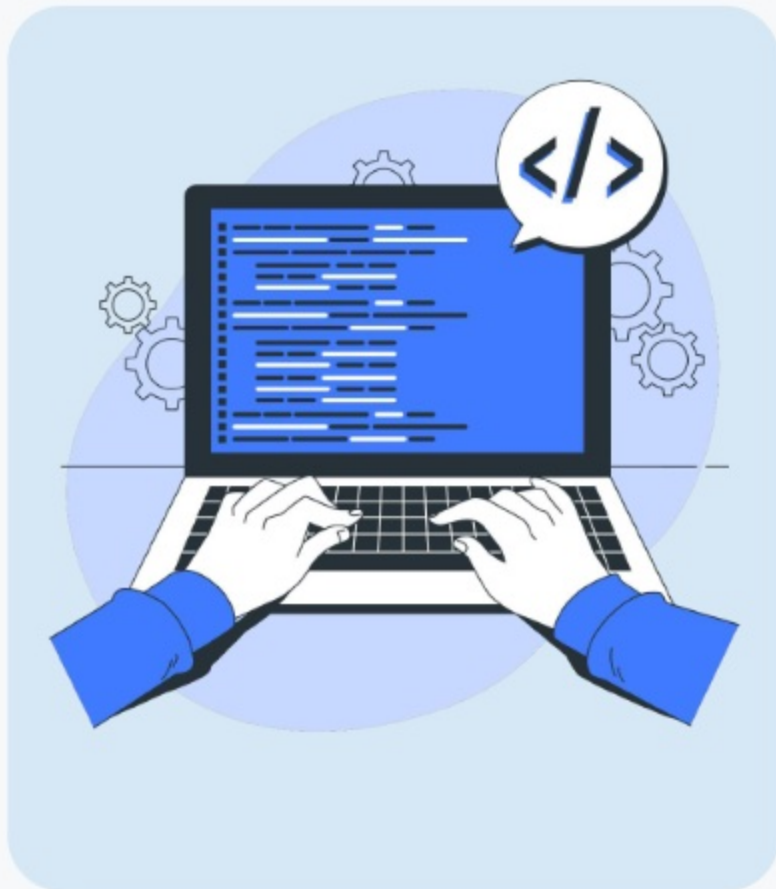
A test data factory contains **reusable methods** for **generating test data**, speeding up development and enhancing the reliability of unit tests.

❖ TOOLS

Various tools can be utilized to manage code quality. These include **source control system like Git**, **Salesforce Extensions for VS Code**, **Salesforce CLI**, and **CI/CD tool**.

Coding Standards

Coding conventions/standards are **guidelines for styling and structuring code** to add consistency and ensure team adherence to the same principles.



❖ USING APEX

Apex code should only be used for **high performance batch processing** needs or **highly sophisticated implementation logic**.

❖ APEX BEST PRACTICES

There are various best practices that apply to the use of Apex code. These include **bulkifying DML statements**, using the **built-in Limits methods**, **writing comments** with a good balance between quality and quantity, **avoiding SOQL queries** and **DML statements within loops**, etc.

❖ USING TDD

Test-Driven Development can be utilized for more modularized, flexible, and extensible code.

Pull Requests

[Table of Contents](#) 

Pull Requests

In Git-based workflows, a pull request is a **proposal to merge changes** from one branch into another.

❖ COMPARISON

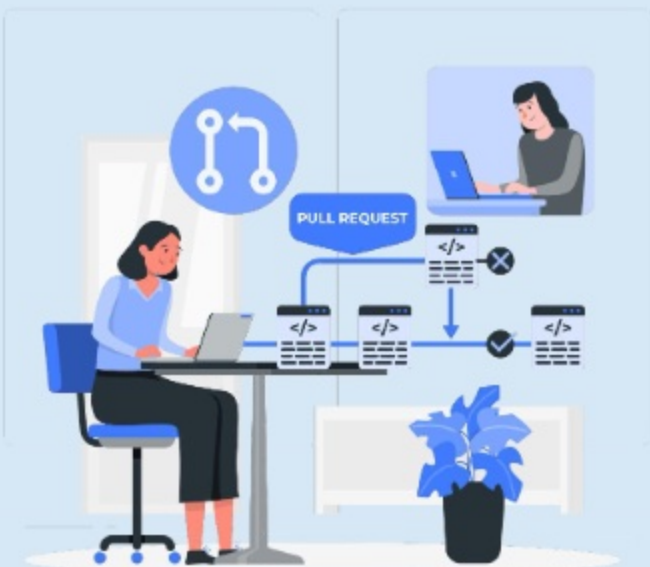
A pull request displays the **differences** between the content in the **source branch** and the content in the **target branch**. It represents a **change to the code (adding, modifying, or deleting files)** on a branch that the author would like to incorporate in another branch.

❖ REVIEW

In a pull request, collaborators can **review** and **discuss the proposed set of changes** before integrating them with the main codebase. **Team members** can be invited to **comment** and **complete code reviews**.

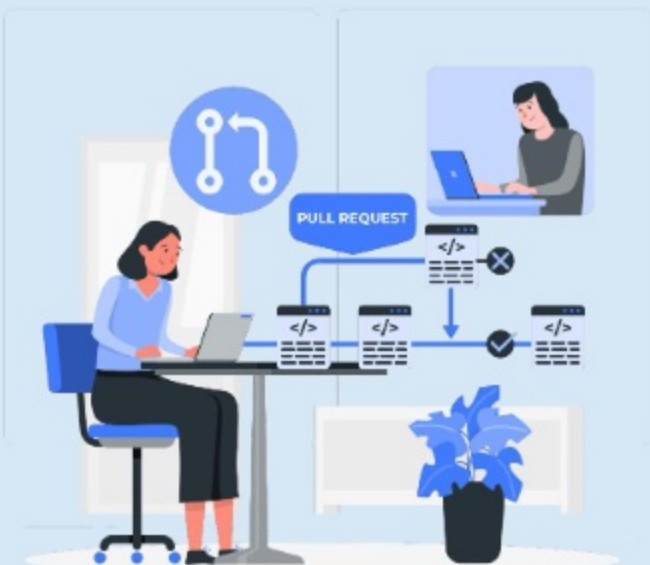
❖ QUALITY & STANDARDS

Every pull request should be reviewed by **at least one other developer** for **code quality** and **adherence to standards**.



Pull Requests

In Git-based workflows, a pull request is a **proposal to merge changes** from one branch into another.



❖ CONVERSATIONS

A pull request gives a platform for having **extended conversations** about the changes between two branches.

❖ AUTOMATED TESTING

Pull requests help write better software by **facilitating code review** and **showing the status of any automated tests**.

❖ INFORMATION

In a pull request, a **summary of the proposed changes** can be added, the **changes made by commits** can be reviewed, **labels, milestones, and assignees** can be added, and **individual contributors or teams** can be @mentioned.

Code Review

[Table of Contents](#) 

Code Review

Code review, also called **peer review**, is a **quality assurance activity** in which one or more people **check the source code** to catch bugs and ensure code quality and adherence to best practices.



❖ PURPOSE

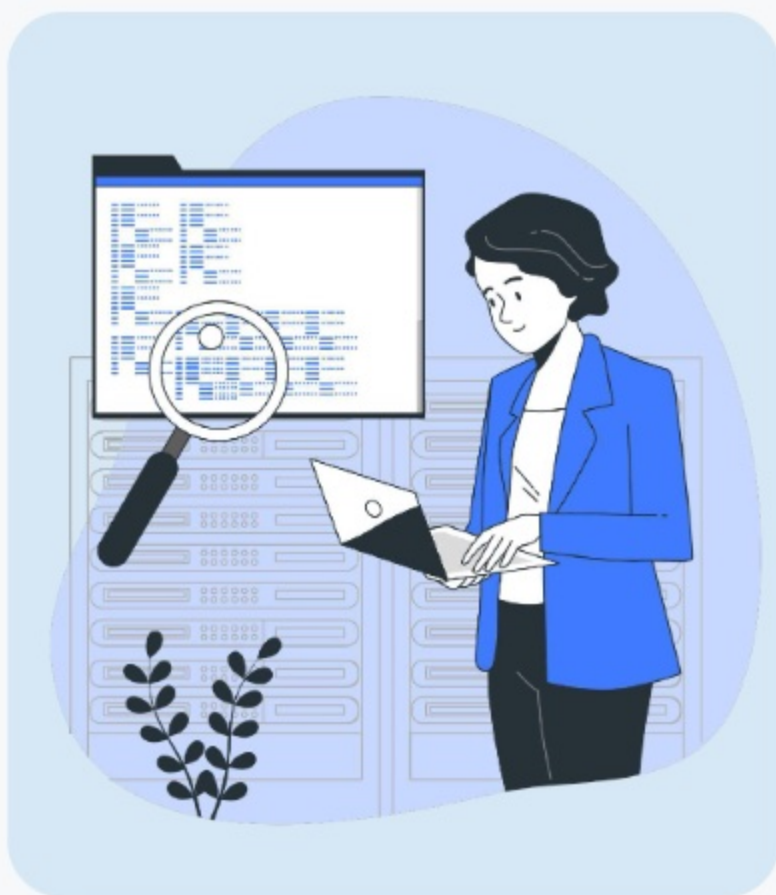
The purpose of code review is to **enhance code quality** and **maintainability**, **detect bugs and issues early**, and **facilitate knowledge sharing** across the development team.

❖ PREPARATION

The **goals** of the code review process should be identified. A **review checklist** should be prepared. Code should be **documented clearly** to explain the logic and decisions.

Code Review

Code review, also called **peer review**, is a **quality assurance activity** in which one or more people **check the source code** to catch bugs and ensure code quality and adherence to best practices.



❖ REVIEW CHECKLIST

A code review should check whether the code follows **standards and best practices**, including the use of naming conventions, correct formatting, and frameworks. Furthermore, the code should be **scalable**, **performant**, **secure**, **easy to understand**, and **easily maintainable**.

❖ EFFECTIVE REVIEWS

Code should be reviewed for **logical correctness**, **efficiency**, and **alignment** with project requirements. It should be checked for compliance with Salesforce best practices, including **governor limits** and **bulkification**. Tools like **GitHub** can be used for facilitating code reviews and discussions.

Code Review

Code review, also called **peer review**, is a **quality assurance activity** in which one or more people **check the source code** to catch bugs and ensure code quality and adherence to best practices.



❖ AUTOMATION & RESOLUTION

The code review process should be **automated** as much as possible using tools such as **static code analysis tools** and **automated testing tools**. If issues are found and need to be resolved, changes should be committed to **source control** and **run through CI/CD processes**.

❖ ISSUES

When evaluating the code in a Salesforce org, some issues that can be expected include **multiple triggers** on the same object, **duplicated logic** across Apex classes and Visualforce pages, existence of **redundant Apex code** that is no longer required due to new Salesforce features, etc.

Code Review

Code review, also called **peer review**, is a **quality assurance activity** in which one or more people **check the source code** to catch bugs and ensure code quality and adherence to best practices.



❖ COMMUNICATION

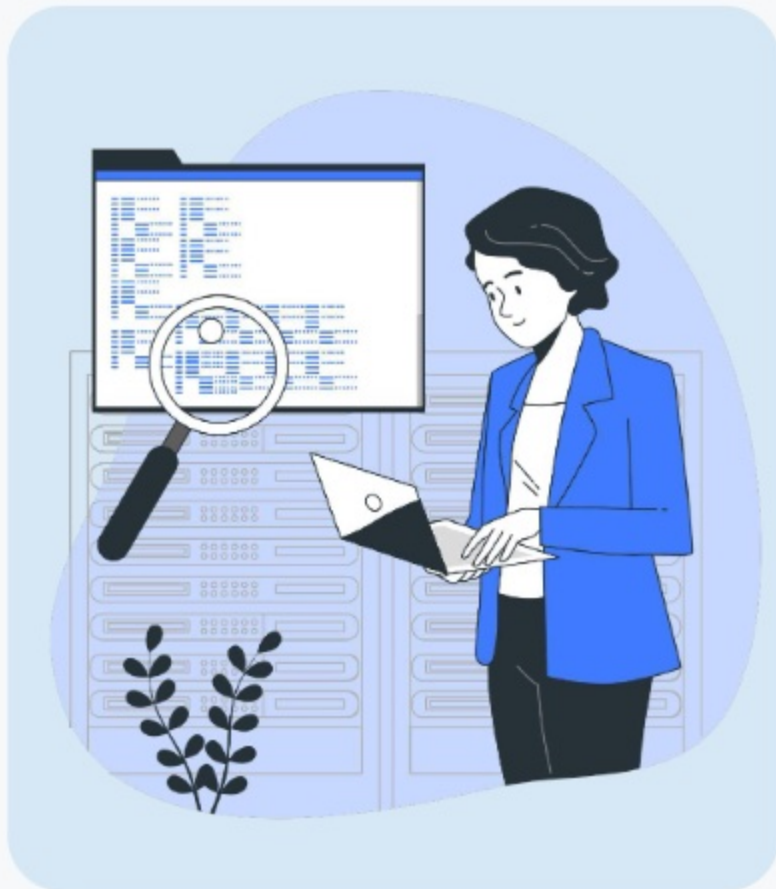
During code reviews, **specific, actionable feedback** focused on code improvement should be provided. All feedback should maintain a **positive and respectful tone**. Developers should be encouraged to explain their **decisions** and **reasoning** when necessary.

❖ INCORPORATING FEEDBACK

Code should be **revised** based on review feedback, **prioritizing critical changes**. **Disagreements** or **misunderstandings** should be discussed and resolved collaboratively. Changes should **only be merged** after all **critical issues** are addressed and the **code meets project standards**.

Code Review

Code review, also called **peer review**, is a **quality assurance activity** in which one or more people **check the source code** to catch bugs and ensure code quality and adherence to best practices.

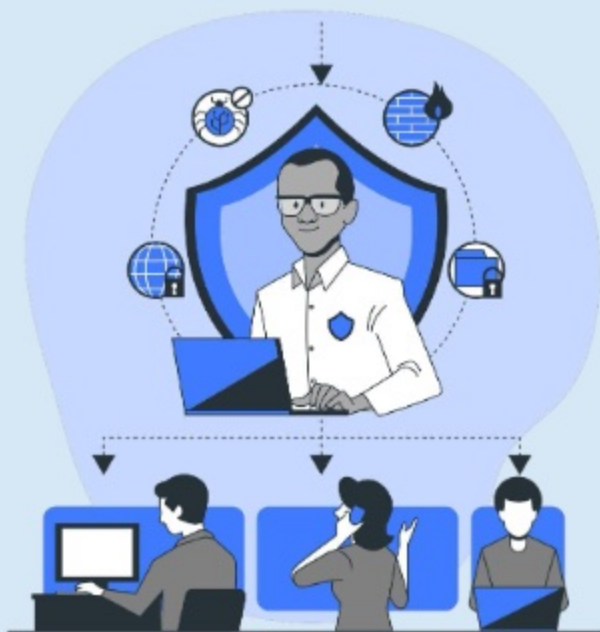


◆ EVALUATION

The effectiveness of the code review process should be **regularly evaluated**. Feedback should be solicited from team members on how to **improve code reviews**. **Changes** should be implemented to the code review process based on **feedback and assessments**.

AppExchange Security Review

The Salesforce security team conducts **reviews** before **green-lighting solutions for the AppExchange**.



❖ PARTNER COMMITMENT

Partners are responsible for leveraging the platform to **protect customer data**, with the **security review process** ensuring **high standards**.

❖ SECURITY THREATS

Partners are responsible for protecting their solutions from the **most common web security threats**, such as injection, sensitive data exposure, session hijacking, and cross-site scripting.

❖ SALESFORCE-SPECIFIC SECURITY

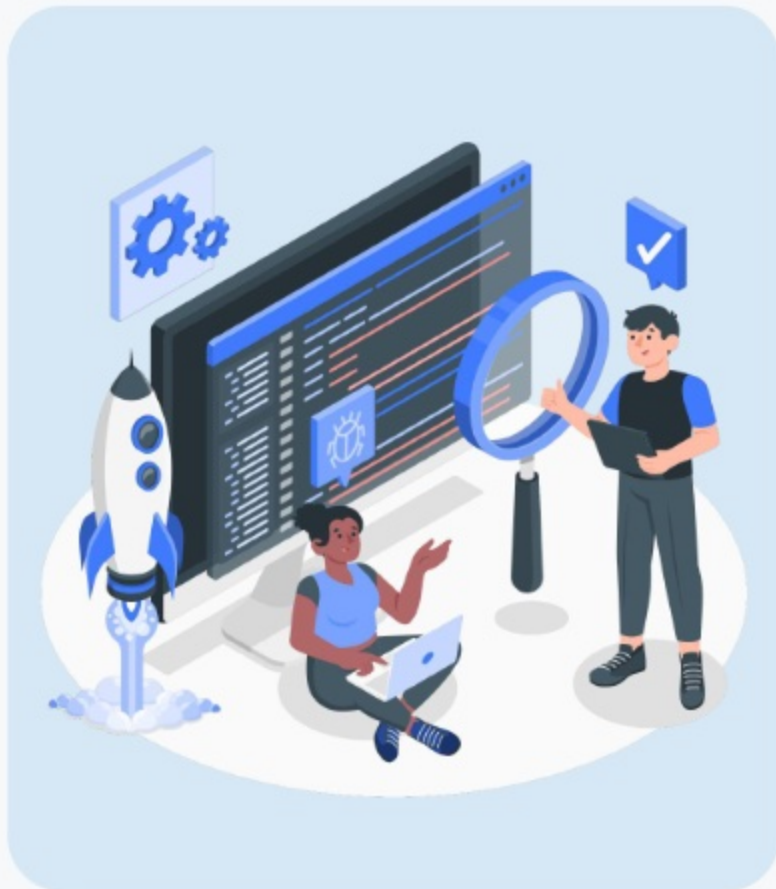
CRUD/FLS must be considered when designing a solution since it is related to how objects interact within the solution. **Failure to properly enforce CRUD/FLS security** is the **most common reason** prospect AppExchange solutions fail the security review.

Static Code Analysis

[Table of Contents](#) 

Static Code Analysis

Static code analysis **examines the code** without executing it to **identify potential vulnerabilities, coding errors**, and **adherence to coding standards**.



❖ IMPORTANCE

Static code analysis ensures **code quality, security**, and **performance** within Salesforce applications.

❖ BENEFITS

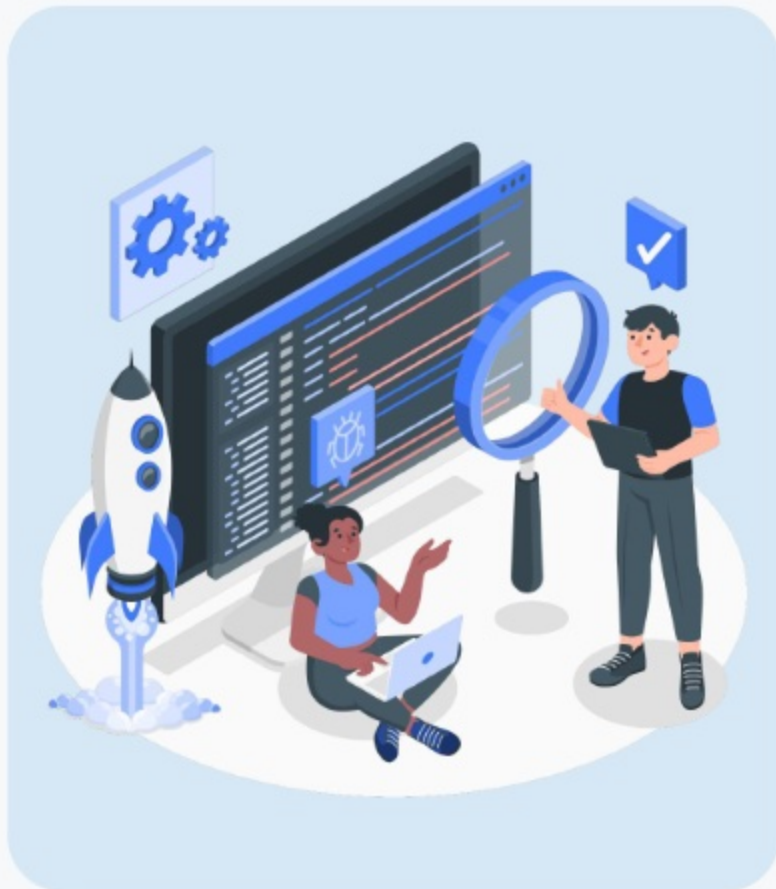
It **detects code issues early** in the development cycle, saving time and reducing costs, **improves code security** by identifying vulnerabilities that could be exploited, and **ensures compliance with Salesforce best practices** and **coding standards**.

❖ TOOLS

Various Salesforce-specific tools are available for static code analysis. These include **PMD, CodeScan, Clayton**, and **Salesforce CLI Scanner**.

Static Code Analysis

Static code analysis **examines the code** without executing it to **identify potential vulnerabilities, coding errors**, and **adherence to coding standards**.



❖ APEX PMD

PMD is an **open-source static code analysis tool** that supports Apex and Visualforce. It finds **common programming flaws** like unused variables, empty catch blocks, unnecessary object creation, and so forth.

❖ CODESCAN

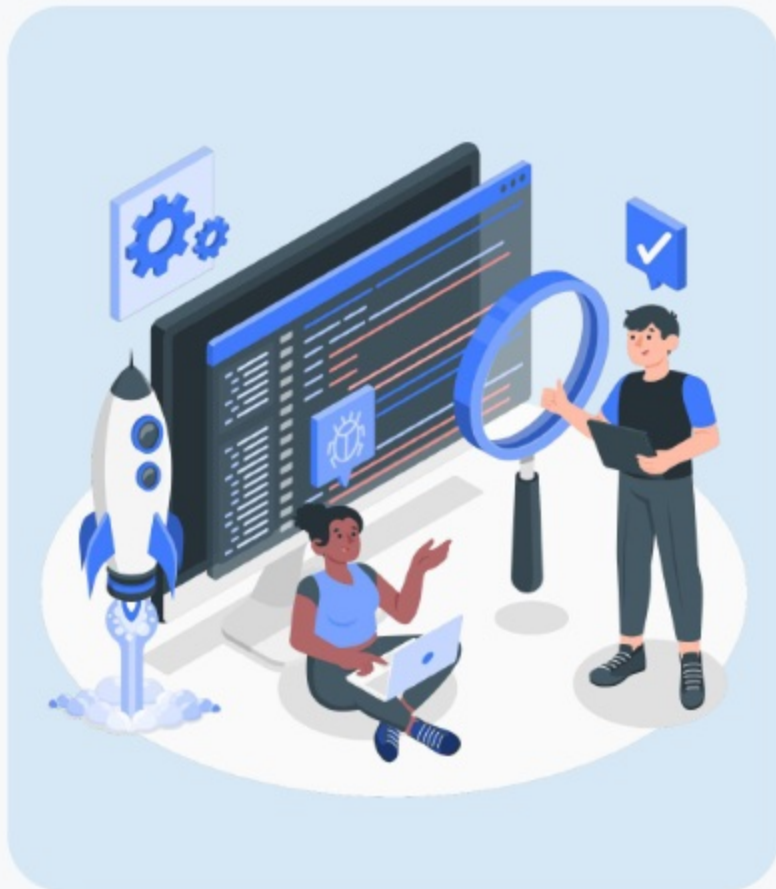
CodeScan is a **comprehensive code analysis tool** specifically designed for Salesforce, covering Apex, Visualforce, and Lightning components.

❖ CLAYTON

Clayton is a **cloud-based code quality tool** tailored for Salesforce, offering automated reviews and coding standards enforcement.

Static Code Analysis

Static code analysis **examines the code** without executing it to **identify potential vulnerabilities, coding errors**, and **adherence to coding standards**.



❖ SALESFORCE CLI SCANNER

Salesforce CLI Scanner is a plugin for **Salesforce CLI** that leverages **PMD** and **ESLint** to scan Apex, Visualforce, Lightning Web Components (LWC), and Aura components.

❖ IMPLEMENTATION

Salesforce DX can be used for a seamless integration of a **static code analysis tool** into the development workflow to routinely check the code.

❖ CONFIGURATION

Static code analysis tools can be **configured to match** the **project's coding standards** and **security policies**.

Static Code Analysis

Static code analysis **examines the code** without executing it to **identify potential vulnerabilities, coding errors**, and **adherence to coding standards**.



❖ REVIEW & ADDRESS

Findings from the static code analysis tool can be **reviewed and addressed** as part of the code review process. The development team can be educated on **common issues** identified by the tool to **prevent them in future coding**.

❖ CI/CD

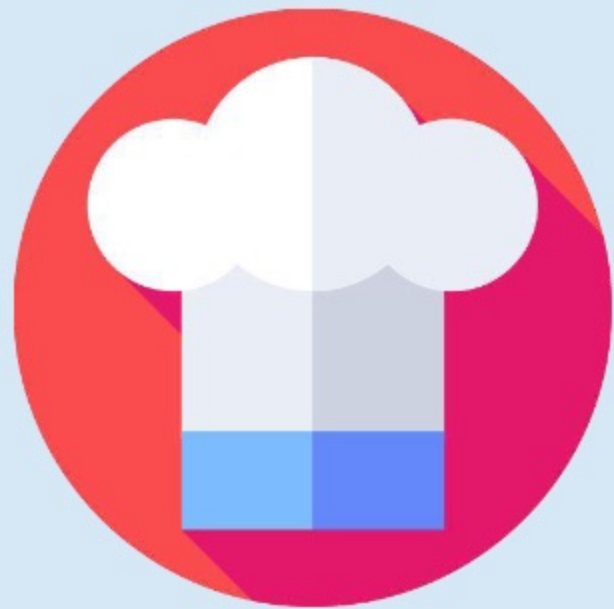
Static code analysis can be **automated as part of CI/CD pipelines** to **ensure consistent code quality** and **security checks** before deployment. Tools like **Jenkins**, **GitLab CI**, and **GitHub Actions** can be configured to run static code analysis tools automatically on **code commits**.

Business Scenario

Table of Contents 

Business Scenario

Read the business scenario below and determine the most appropriate solutions for the company's requirements.



Cosmic Gourmet Solutions, a leading global provider in the food industry, with operations spanning the US, France, Italy, and Japan, leverages Salesforce to manage its intricate network of gourmet product sales, supply chain logistics, and customer engagement. As it embarks on an ambitious project to enhance its Salesforce applications—aiming to introduce new features, swiftly address bugs, and implement updates—the complexity of its development processes surges. Recognizing the need for specialized expertise, the company seeks the guidance of a Salesforce architect. This expert needs to facilitate the adoption of best practices in coding standards, pull requests, code reviews, and static code analysis, ensuring the delivery of quality code and proper maintenance of the existing codebase.

Requirements & Solutions

[Table of Contents](#) 

Requirement 1



SCENARIO

The company has multiple teams working on different projects. During code reviews, several inconsistently named variables and lack of best practices have been identified. The number of bugs has been increasing after each sprint. They need more time to understand the code and make changes after each sprint, which often delays the delivery of new features and updates. The architect must recommend methods and practices to face these challenges and improve the performance of the teams.

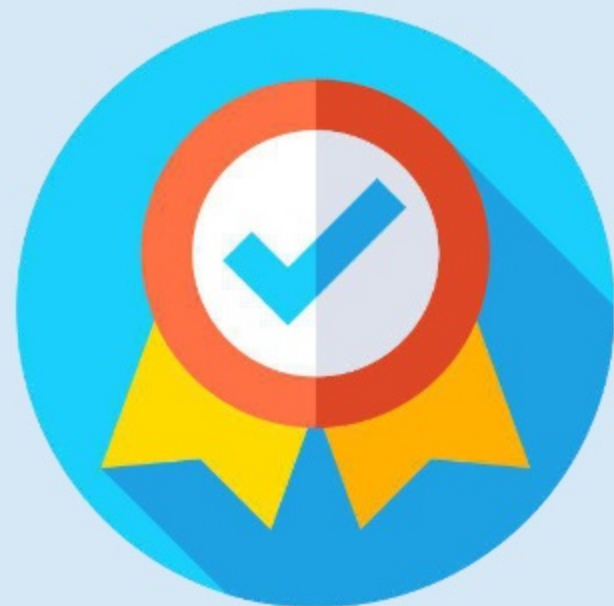


Solution 1



SOLUTION

In order to improve code quality and consistency, the teams should **define** and **follow coding standards**. The standards should be enforced using a **static code analysis tool**. The performance of the teams can be improved by implementing a **sprint review process** that involves code reviews. The static code analysis tool should be used before the manual reviews. Also, **development standards** should be created, and the teams should be **trained** in those standards.

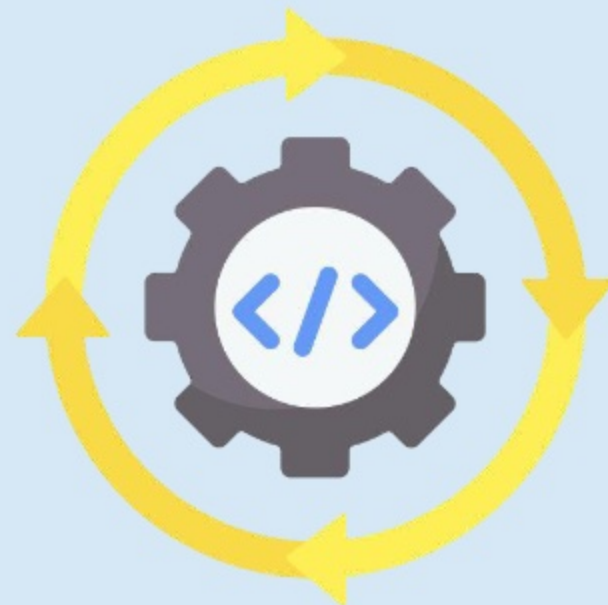


Requirement 2



SCENARIO

Most of the development teams have been using a CI/CD pipeline. The CTO would like to incorporate quality assurance techniques into the CI/CD processes to minimize defects in the applications, improve code quality, and also improve code security. In addition, the company has acquired code review software licenses to help the developers with code quality. The architect needs to recommend practices for secure code reviews.



Solution 2



SOLUTION

Static code analysis and **automated testing** should be incorporated into the CI/CD pipelines to **minimize defects** in the applications. Before deployment to the testing sandbox, **static code analysis** and **manual code review** should be performed. For the improvement of code security, **pull requests** can be utilized for secure code reviews. **Human efforts** can be combined with **automated checks** by the code review software. A **code review checklist** should be generated to ensure consistency of reviews.

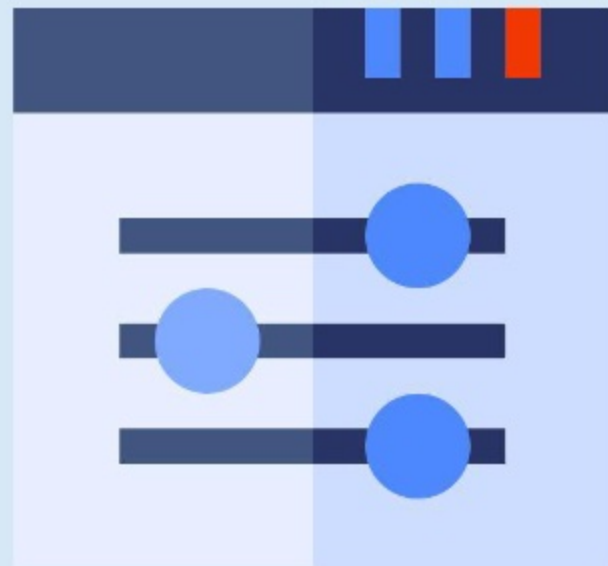


Requirement 3



SCENARIO

The company's employees in France use a separate Salesforce org to manage its operations. It was implemented several years ago by a Salesforce Systems Integration Partner. Since then, various changes have been made to its Apex classes and Visualforce pages in response to customer requirements. The architect needs to evaluate the code and identify issues that should be addressed. In addition, the company wants to build a continuous integration process for the org to help manage code quality. All the developed code should be reviewed, tested, and validated in the upstream deployment process.

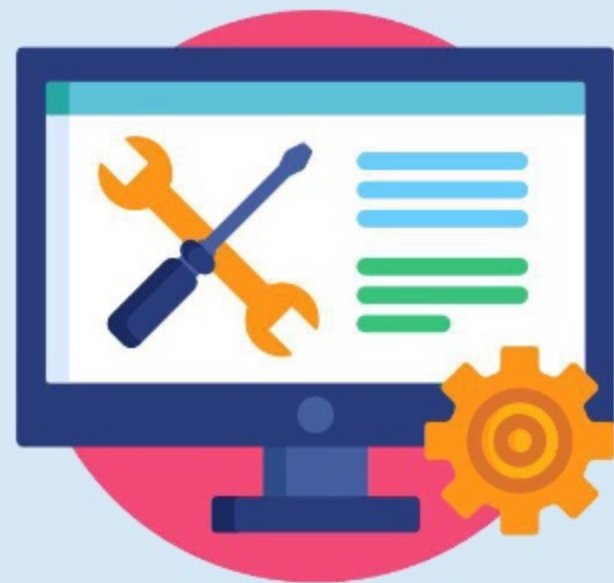


Solution 3



SOLUTION

When evaluating the code in the org, the architect can expect various code quality issues, such as **multiple triggers** on the same object, **duplicated logic** across Apex classes and Visualforce pages, and **obsolete Apex classes** that are no longer required due to the introduction of new functionality by Salesforce. For the continuous integration process, a **source control system** such as Git, a **CI/CD tool**, **Salesforce Extensions for VS Code**, and **Salesforce CLI** can be utilized. **Automated testing** and **test-driven development** should be incorporated into the process. Development teams should perform **code reviews** to improve code quality.



Requirement 4



SCENARIO

The company has started working with a Salesforce partner to develop a Lightning Bolt solution for food product management for the AppExchange marketplace. The solution will be submitted for security review. The architect must provide any related considerations.



Solution 4



SOLUTION

Before submitting the solution for security review, the partner must ensure that the solution is **protected** from the **most common web security threats**, such as injection, sensitive data exposure, session hijacking, and cross-site scripting. **CRUD/FLS** must be considered since it is related to how objects interact within the solution. **Failure to properly enforce CRUD/FLS** security is the most common reason prospect AppExchange solutions fail the security review.



Learn More

-  [Success Cloud Naming Conventions](#)
-  [Salesforce Naming Conventions](#)
-  [Pull Requests](#)
-  [Salesforce Code Review Checklist](#)
-  [AppExchange Security Review](#)
-  [Learn Secure Development Best Practices](#)

